

MGMT 803, MBA@RICE, 2026

Building agents

Kerry Back

J. Howard Creekmore Professor of Finance

This session

- 13. How software works
- 14. API calls
- 15. Build a chatbot
- 16. Build an agent

13 · HOW SOFTWARE WORKS

Frontend and backend

FRONTEND

The browser. HTML, CSS and JavaScript — it draws the page and collects the clicks.

You open a URL. The browser sends a request. The server sends back HTML, CSS and JavaScript. The browser interprets that and shows you a page.

Right-click any page and choose View Page Source. That is the frontend code, exactly as the backend handed it over.

BACKEND

A program on a server, listening continuously for requests. Yours will be Python.

Two kinds of request

	WHAT IT CARRIES	WHAT THE SERVER DOES
GET	a path, and parameters in the URL	sends back a page, or data
POST	a body — a form, a ticker you typed	whatever it was written to do, then replies

Opening a page is a GET. Clicking submit is usually a POST.

The frontend and the backend can be the same computer — your container, or your own laptop with Python on it.

localhost

`127.0.0.1` is the loopback address. The browser asks the machine it is already running on, and the request never reaches a network.

Ports

One machine runs many programs listening at once. The port number tells the operating system which of them gets the request.

You write the app as a `.py` file. FastAPI is the usual Python library for one, and `uvicorn` starts it listening: `uvicorn app:app --host 0.0.0.0 --port 8000`.

Running it on one machine

- 1 Ask for a FastAPI app with one page and one input — a unit converter, a loan payment, anything with a number in and a number out.
- 2 Ask Claude to run it, and open the View tab. Click Refresh.
- 3 Watch the terminal while you use it. Opening the page logs a `GET`; submitting the form logs a `POST`.

Build something interactive

Single-page HTML apps

- There is an exception to the frontend/backend pattern.
- If an app gets its data from browser input and does not need to reach outside, then we may not need a backend server. The code execution (JavaScript) can be done in the browser.

- 1 Ask for the same app as a single-page HTML app.
- 2 When it is complete, click Refresh in the File Explorer and you should see a new .html file.
- 3 Download the file.
- 4 In your File Explorer (Finder on Mac) — **not in your browser** — double-click the file. It should open in your browser. Test it.

Artifacts

Claude calls a single-page HTML app an artifact.

- 1 In Claude Desktop, choose Artifact in the left pane.
- 2 Choose New. Prompt Claude to create the same app.
- 3 You should get options to deploy the app online or to download it.

■

Breakout exercise

Get free accounts at Replit and Lovable and build the same app with their tools. Then try something more ambitious (you may run out of credits on the free tier).

14 · API CALLS

APIs

- An API (application programming interface) is a feature of a software service that allows other services to talk to it
- For example, if I want my website to accept payment by PayPal, then my website has to talk to PayPal's API
- If I want my program to send messages and get responses from an LLM, then my program has to talk to the LLM provider's API

An API call in Python

```
1 import anthropic
2
3 client = anthropic.Anthropic()          # reads ANTHROPIC_API_KEY
4
5 SYSTEM = "Classify the complaint: damage, delay, billing, or other. One word."
6
7 resp = client.messages.create(
8     model="claude-sonnet-5",
9     max_tokens=500,
10    system=SYSTEM,
11    messages=[{"role": "user", "content": "The unit arrived cracked."}],
12 )
13
14 print(resp.content[0].text)
```

The same call through OpenRouter

```
1 from openai import OpenAI
2
3 client = OpenAI(                                # reads OPENAI_API_KEY
4     base_url="https://openrouter.ai/api/v1",
5 )
6 resp = client.chat.completions.create(
7     model="nvidia/nemotron-3.5-lightning:free",
8     messages=[
9         {"role": "system", "content": SYSTEM},
10        {"role": "user", "content": "The unit arrived cracked."},
11    ],
12 )
13 print(resp.choices[0].message.content)
```

The system prompt is a message here, not a separate argument. A different library, a different response shape, and the model is a string you can change. Fifty free requests a day.

What comes back

ANTHROPIC

```
1 {
2   "role": "assistant",
3   "content": [
4     {"type": "text", "text": "damage"}
5   ],
6   "stop_reason": "end_turn",
7   "usage": {"input_tokens": 38,
8             "output_tokens": 4}
9 }
```

OPENROUTER

```
1 {
2   "choices": [
3     {"message": {"role": "assistant",
4                  "content": "damage"},
5      "finish_reason": "stop"}
6   ],
7   "usage": {"prompt_tokens": 38,
8             "completion_tokens": 4}
9 }
```

Both carry the word `damage`. The envelope differs, which is why one prints `resp.content[0].text` and the other `resp.choices[0].message.content`. `stop_reason` and `finish_reason` are how the app knows whether the model finished or wants to use a tool.

Get two keys

ANTHROPIC

`platform.claude.com` → API keys →
Create key. Pay per token, so set a
spending limit on it.
Store it as `ANTHROPIC_API_KEY`.

OPENROUTER

`openrouter.ai` → Keys → Create key.
Sign in with Google or create an account
with username and password, no card.
One key reaches every provider it carries.
Store it as `OPENAI_API_KEY`. The
OpenAI library looks for that name only.

The keys are only shown once. Copy each into a file called `env.txt`, one `NAME=value` per line, and upload it to your container from the Files tab. Then tell Claude: *my keys are in `env.txt`*.

15 · BUILD A CHATBOT

FastAPI app that makes API calls

- A chatbot is a software program that calls an LLM.
- We can build one as a FastAPI app.
- The app should have a user interface (prompt window).
- The browser sends prompts to the backend as POST requests.
- The backend makes an API call, gets a response, and replies to the browser.

What does it need?

- An endpoint for the API call (Anthropic, OpenAI, OpenRouter, ...)
- A model name
- A system prompt
- A loop to keep the message list
- A user interface

We need to tell Claude what provider we want (Anthropic, OpenAI, ...), what model we want, and what the system prompt should be. Claude can do everything else.

- 1 Build a chatbot on either key — Anthropic (ANTHROPIC_API_KEY) or OpenRouter (OPENAI_API_KEY), your choice.
- 2 Put your name on it, where anyone using it can see.
- 3 Give it a system prompt that makes it answer in a distinctive way (for example, talk like a pirate).

You can tell Claude to do these things. You are not expected to write the code yourself.

Breakout exercise

16 · BUILD AN AGENT

Chatbot vs agent

The chatbot

Text in, text out. The app does the same thing with every reply it gets

The agent

The reply can be a request to run something. The app runs it, sends the result back, and asks the model what to do next

An agent is an app that takes actions at the discretion of a model.

- 1 You send a request. The harness also sends a list of tools and what each one does.
- 2 The model answers with text, or with a request to use a tool.
- 3 The harness runs the tool and puts the result back into the conversation.
- 4 Repeat until the model answers with text and no tool call.

An agent is a chatbot with tools

A tool is a description

```
1 {
2   "tools": [
3     {"name": "query_northwind",
4      "description": "Run a read-only SELECT against Northwind. Returns CSV.",
5      "input_schema": {
6        "type": "object",
7        "properties": {"query": {"type": "string"}},
8        "required": ["query"]}
9   ]
10 }
```

Sent in the same request, alongside `messages`. The description is what the model reads when deciding whether to call it.

The reply asks for the tool

```
1 {
2   "role": "assistant",
3   "content": [
4     {"type": "tool_use",
5      "id": "toolu_01A9pK",
6      "name": "query_northwind",
7      "input": {
8        "query": "SELECT Country, COUNT(*) FROM customers GROUP BY Country"
9      }}
10  ],
11  "stop_reason": "tool_use"
12 }
```

Your code runs the query and calls again, with a `tool_result` block carrying the same `id`, and the whole conversation behind it.

Three kinds of tools

	WHERE THE CODE IS	EXAMPLE
Built in	in the harness	read a file, run a command, fetch a page
Yours	a function you wrote	look up a customer in your system
Someone else's	an MCP server	the Northwind database, EIA, FRED

All three arrive at the model the same way: a name, a description, and a parameter schema.

Tools from an agent SDK

Pre-built code for the harness, and a set of tools that come with it. The Claude Agent SDK, the OpenAI Agents SDK, LangChain.

Read	Write	Edit
Glob	Grep	Bash
WebSearch	WebFetch	Task

The built-ins in the Claude Agent SDK. You choose which to include: `tools=[]` includes none, and naming a few includes only those. **If we use the Claude Agent SDK, we get the Claude Code harness.**

Tools from MCP servers

Tools written by somebody else, running on a server somebody else operates. Thousands of MCP servers exist already.

What you add

A URL, in the agent's configuration

What arrives

A name, a description and an input schema for every tool the server publishes

What stays there

The code, and the credential it uses to reach the service

To the model these are more entries in the same tool list. What differs is whose machine runs them.

Custom tools

```
1 @tool(  
2     "add_two_numbers",  
3     "Add two numbers and return the sum.",  
4     {"a": Annotated[float, "The first number"],  
5      "b": Annotated[float, "The second number"]},  
6 )  
7 async def add_two_numbers(args):  
8     total = args["a"] + args["b"]  
9     return {"content": [{"type": "text", "text": str(total)}]}
```

A name, a description and typed arguments — the three parts every tool has — written as a Python function. The SDK turns the decorator into the JSON it sends the model.

- 1 Build an agent using the Claude Agent SDK and your Anthropic API key.
- 2 Give it the FRED MCP server, `fred.kerryback.com/mcp`, as its only source of data.
- 3 Give it a `run_python` tool.
- 4 Run it and ask for something that needs data and python: for example, unemployment and inflation over the last ten years, on one chart.
- 5 Repeat, but instead of using the Claude Agent SDK, tell Claude you want to use the OpenCode harness and your OpenRouter API key.

Breakout exercise